

Technical Architecture Document

Product: Planetic Solution Automated Website, Domain, Hosting, Cloudflare DNS & Billing Platform

1. Executive Summary

This document defines the recommended technical architecture for the Planetic Solution platform.

The platform will allow customers to:

- Buy a complete bespoke website package for £200
- Get free first-year domain and hosting with selected packages
- Search and register domains
- Buy hosting packages
- Pay online
- Get hosting automatically provisioned in Namecheap WHM
- Get DNS automatically configured in Cloudflare
- View services, invoices, renewals, and payment status
- Submit website project details
- Submit support tickets

The platform will also allow Planetic Solution admins to:

- Manage customers
- Manage orders
- Manage payments
- Manage domain registrations
- Manage hosting accounts
- Manage Cloudflare DNS setup
- Track website development projects
- Handle failed provisioning jobs
- Handle renewals, suspensions, and support tickets

The recommended architecture is a Laravel-based monolithic application with modular service classes for Stripe, WHM, Cloudflare, and registrar APIs.

This approach gives the fastest development path, easier deployment on cPanel/VPS, strong database control, queue support, admin dashboard capability, and long-term scalability.

2. Recommended Tech Stack

2.1 Backend Framework

Recommended: Laravel

Laravel should be used as the main backend framework.

Why Laravel

Laravel is recommended because this project needs:

- User authentication
- Customer dashboard
- Admin dashboard
- Payment workflows
- API integrations
- Database-driven business logic
- Scheduled renewal checks
- Queue-based provisioning jobs
- Email notifications
- Secure environment configuration
- Role-based admin access

Laravel is especially suitable because this platform is not only a marketing website. It is a transactional system that manages payments, customers, hosting, domains, DNS, invoices, and automation.

Alternative Options

Option	Suitability	Reason
Laravel	Excellent	Best balance of speed, structure, queues, billing, admin, and database workflows
Next.js + Node.js	Good	Strong for frontend, but more setup needed for billing/admin/business workflows
WordPress custom plugin	Weak for this system	Suitable for marketing pages, not ideal for automation-heavy billing platform
WHMCS only	Good for hosting billing, weak for custom UX	Less flexible if you want your own branded dashboard and custom checkout
Django	Good	Strong but less aligned with your current cPanel/PHP hosting environment

2.2 Frontend

Recommended: Laravel Blade + Tailwind CSS + Alpine.js

For the MVP, use Laravel Blade with Tailwind CSS and Alpine.js.

Why

This keeps the project simple and fast to build. You do not need a separate frontend app in the beginning. Laravel Blade is enough for:

- Marketing pages
- Pricing pages
- Checkout pages
- Customer dashboard
- Admin dashboard
- Forms
- Tables
- Status pages

Tailwind CSS gives professional UI styling without heavy custom CSS.

Alpine.js can handle lightweight interactivity such as dropdowns, modals, tabs, package toggles, and dynamic form sections.

Future Option

Later, if needed, the customer dashboard can be converted to Vue, React, or Inertia.js. But for the MVP, do not overcomplicate it.

2.3 Database

Recommended: MySQL 8+

Use MySQL because:

- It is widely supported on cPanel and VPS hosting
- Laravel has excellent MySQL support
- The data model is relational
- Orders, invoices, domains, hosting accounts, users, and subscriptions have clear relationships
- It is easier for deployment in your current hosting environment

2.4 Queue System

MVP Recommended: Database Queue

For the MVP, use Laravel database queues.

This allows background jobs for:

- Domain registration
- WHM hosting account creation
- Cloudflare zone creation
- DNS record creation
- Email sending
- Renewal reminders

- Failed payment processing

Later Recommended: Redis + Laravel Horizon

When the platform grows, move to Redis queues and Laravel Horizon for better queue monitoring.

2.5 Payments

Recommended: Stripe

Stripe should be used for:

- One-time payments
- Hosting subscriptions
- Domain renewal payments
- Payment method management
- Failed payment handling
- Invoice payment tracking
- Webhook-based billing automation

Payment Model

Use Stripe Checkout for the MVP because it is faster, secure, and reduces PCI complexity.

Later, you can use embedded Stripe Elements if you want the full payment form inside your website.

2.6 Hosting Provisioning

Recommended: Namecheap Reseller WHM API

Use WHM API to:

- Create cPanel accounts
- Assign packages
- Suspend accounts
- Unsuspend accounts
- Terminate accounts if needed
- List hosting accounts
- Check account status

Hosting package names should match WHM package names.

Example:

Website Package	WHM Package
Starter	planetic_starter
Business	planetic_business
Pro	planetic_pro

Website Package	WHM Package
Agency / Ecommerce	planetic_agency

2.7 Domain Registrar

Recommended: NameSilo API

NameSilo should be the first-choice registrar API because it is well suited for automation and domain reseller workflows.

Use it for:

- Domain availability checks
- Domain registration
- Domain renewal
- Domain information
- Expiry date retrieval
- Nameserver updates
- WHOIS privacy where available
- Domain list sync

Backup Option: Namecheap Domain API

Namecheap Domain API can be used as a secondary provider. It can also register domains and set custom nameservers.

Porkbun

Do not use Porkbun as the primary automation provider unless you confirm full API support for domain registration, renewal, contact management, and nameserver updates.

2.8 DNS Provider

Recommended: Cloudflare API

Cloudflare will be the central DNS and security provider.

Use Cloudflare for:

- DNS zone creation
- DNS record management
- Nameserver assignment
- SSL/TLS settings
- Always Use HTTPS
- CDN/proxy for website records
- DNS-only mode for mail/cPanel-related records

2.9 Email Provider

Recommended for MVP

Use SMTP through a reliable provider such as:

- Mailgun
- SendGrid
- Amazon SES
- Resend
- SMTP2GO

Do not rely only on basic cPanel mail for transactional emails. Payment, renewal, and provisioning emails are important and must be reliable.

2.10 Admin Panel

Recommended: Filament Admin Panel for Laravel

Filament is recommended because it allows fast admin dashboard development with Laravel models.

Use it for:

- Customer management
- Order management
- Domain management
- Hosting account management
- Website project management
- Support ticket management
- Provisioning logs
- Payment records

2.11 Deployment

MVP Deployment Options

Option 1: VPS

Best for long-term control.

Option 2: cPanel Laravel deployment

Possible, but queue workers and cron must be configured carefully.

Recommended production setup:

- Ubuntu VPS
- Nginx
- PHP-FPM
- MySQL
- Redis later
- Supervisor for queue workers

- Laravel scheduler cron
- SSL certificate
- GitHub deployment

If starting from cPanel, make sure you can run:

- Composer install
 - PHP Artisan commands
 - Cron jobs
 - Queue workers
 - Environment variables
 - Storage symlink
-

3. High-Level System Architecture

3.1 Main Components

The system will contain the following modules:

1. Public Website Module
2. Authentication Module
3. Customer Dashboard Module
4. Admin Dashboard Module
5. Product and Pricing Module
6. Cart and Checkout Module
7. Stripe Billing Module
8. Domain Registrar Module
9. WHM Hosting Module
10. Cloudflare DNS Module
11. Provisioning Engine
12. Renewal Engine
13. Notification Engine
14. Website Project Module
15. Support Ticket Module
16. Audit and Logging Module

3.2 System Flow

Main flow:

1. Customer chooses website/domain/hosting package.
2. Customer creates account.
3. Customer pays through Stripe.
4. Stripe sends webhook to Laravel.
5. Laravel validates payment.
6. Laravel creates order and provisioning job.
7. Provisioning job registers domain.
8. Provisioning job creates Cloudflare zone.
9. Provisioning job updates registrar nameservers to Cloudflare.

10. Provisioning job creates WHM hosting account.
11. Provisioning job creates DNS records in Cloudflare.
12. Laravel sends welcome emails.
13. Customer sees services in dashboard.
14. Admin can monitor everything from admin panel.

3.3 Integration Boundaries

Each third-party system must be wrapped inside a dedicated Laravel service class.

Examples:

- StripeService
- NameSiloService
- NamecheapRegistrarService
- WhmService
- CloudflareService
- EmailNotificationService

The controller should never call external APIs directly.

Correct pattern:

Controller

→ Application Action / Service

→ Job

→ Integration Service

→ Database Update

→ Notification

4. Recommended Laravel Project Structure

The project should follow a clean Laravel structure with extra folders for business modules and third-party integrations.

```
planetic-platform/  
|  
├─ app/  
│   └─ Actions/  
│       └─ Checkout/  
│           ├── CreateCheckoutSession.php  
│           └─ CompletePaidOrder.php  
│       └─ Domains/  
│           ├── CheckDomainAvailability.php  
│           ├── RegisterDomain.php  
│           └─ RenewDomain.php
```

- └─ UpdateDomainNameservers.php
- └─ Hosting/
 - └─ CreateHostingAccount.php
 - └─ SuspendHostingAccount.php
 - └─ UnsuspendHostingAccount.php
 - └─ SyncHostingAccount.php
- └─ Cloudflare/
 - └─ CreateCloudflareZone.php
 - └─ CreateDefaultDnsRecords.php
 - └─ EnableDefaultCloudflareSettings.php
 - └─ SyncCloudflareStatus.php
- └─ Orders/
 - └─ CreateOrderFromCart.php
 - └─ MarkOrderAsPaid.php
 - └─ MarkOrderAsFailed.php
 - └─ CancelOrder.php
- └─ Renewals/
 - └─ GenerateRenewalReminder.php
 - └─ ProcessSuccessfulRenewal.php
 - └─ ProcessFailedRenewal.php
- └─ WebsiteProjects/
 - └─ CreateWebsiteProject.php
 - └─ UpdateWebsiteProjectStatus.php
 - └─ RequestWebsiteContent.php
- └─ Console/
 - └─ Commands/
 - └─ CheckRenewalsCommand.php
 - └─ SyncDomainStatusesCommand.php
 - └─ SyncHostingStatusesCommand.php
 - └─ RetryFailedProvisioningCommand.php
- └─ Enums/
 - └─ DomainStatus.php
 - └─ HostingStatus.php
 - └─ InvoiceStatus.php
 - └─ OrderStatus.php
 - └─ PaymentStatus.php
 - └─ ProvisioningStatus.php
 - └─ SubscriptionStatus.php
 - └─ SupportTicketStatus.php
 - └─ WebsiteProjectStatus.php
- └─ Events/
 - └─ OrderPaid.php

- └─ DomainRegistered.php
- └─ HostingAccountCreated.php
- └─ CloudflareZoneCreated.php
- └─ PaymentFailed.php
- └─ ServiceRenewalDue.php
- ─ Exceptions/
 - └─ RegistrarException.php
 - └─ WhmException.php
 - └─ CloudflareException.php
 - └─ ProvisioningException.php
- ─ Filament/
 - └─ Resources/
 - └─ UserResource.php
 - └─ OrderResource.php
 - └─ DomainResource.php
 - └─ HostingAccountResource.php
 - └─ SubscriptionResource.php
 - └─ InvoiceResource.php
 - └─ WebsiteProjectResource.php
 - └─ SupportTicketResource.php
 - └─ ProvisioningJobResource.php
 - └─ Pages/
 - └─ Dashboard.php
 - └─ RevenueReport.php
 - └─ ProvisioningMonitor.php
- ─ Http/
 - └─ Controllers/
 - └─ Public/
 - └─ HomeController.php
 - └─ PricingController.php
 - └─ DomainSearchController.php
 - └─ ContactController.php
 - └─ Customer/
 - └─ DashboardController.php
 - └─ DomainController.php
 - └─ HostingController.php
 - └─ BillingController.php
 - └─ WebsiteProjectController.php
 - └─ SupportTicketController.php
 - └─ CheckoutController.php
 - └─ Webhook/
 - └─ StripeWebhookController.php
 - └─ Middleware/

- └─ EnsureUserIsAdmin.php
- └─ EnsureEmailVerified.php
- └─ VerifyStripeWebhook.php
- └─ Requests/
 - └─ CheckoutRequest.php
 - └─ DomainSearchRequest.php
 - └─ WebsiteProjectIntakeRequest.php
 - └─ SupportTicketRequest.php
 - └─ UpdateBillingDetailsRequest.php
- └─ Jobs/
 - └─ Provisioning/
 - └─ ProvisionOrderJob.php
 - └─ RegisterDomainJob.php
 - └─ CreateCloudflareZoneJob.php
 - └─ UpdateRegistrarNameserversJob.php
 - └─ CreateWhmHostingAccountJob.php
 - └─ CreateCloudflareDnsRecordsJob.php
 - └─ SendProvisioningCompletedEmailJob.php
 - └─ Billing/
 - └─ HandleStripeInvoicePaidJob.php
 - └─ HandleStripePaymentFailedJob.php
 - └─ SyncStripeSubscriptionJob.php
 - └─ Renewals/
 - └─ SendRenewalReminderJob.php
 - └─ SuspendOverdueHostingJob.php
 - └─ UnsuspendPaidHostingJob.php
 - └─ Notifications/
 - └─ SendWelcomeEmailJob.php
 - └─ SendOrderConfirmationEmailJob.php
 - └─ SendSupportTicketEmailJob.php
- └─ Listeners/
 - └─ StartProvisioningAfterPayment.php
 - └─ NotifyAdminOfProvisioningFailure.php
 - └─ SendCustomerPaymentReceipt.php
- └─ Mail/
 - └─ OrderConfirmationMail.php
 - └─ PaymentFailedMail.php
 - └─ HostingWelcomeMail.php
 - └─ DomainRegisteredMail.php
 - └─ RenewalReminderMail.php
 - └─ SupportTicketUpdatedMail.php
- └─ Models/

- └─ User.php
- └─ Role.php
- └─ Product.php
- └─ ProductPrice.php
- └─ Cart.php
- └─ CartItem.php
- └─ Order.php
- └─ OrderItem.php
- └─ Payment.php
- └─ Invoice.php
- └─ Subscription.php
- └─ Domain.php
- └─ DomainContact.php
- └─ HostingAccount.php
- └─ CloudflareZone.php
- └─ DnsRecord.php
- └─ ProvisioningJob.php
- └─ WebsiteProject.php
- └─ WebsiteProjectAsset.php
- └─ SupportTicket.php
- └─ SupportTicketMessage.php
- └─ NotificationLog.php
- └─ AuditLog.php
- └─ Notifications/
 - └─ RenewalDueNotification.php
 - └─ PaymentFailedNotification.php
 - └─ HostingSuspendedNotification.php
 - └─ WebsiteProjectUpdatedNotification.php
- └─ Policies/
 - └─ DomainPolicy.php
 - └─ HostingAccountPolicy.php
 - └─ InvoicePolicy.php
 - └─ SupportTicketPolicy.php
 - └─ WebsiteProjectPolicy.php
- └─ Providers/
 - └─ AppServiceProvider.php
 - └─ EventServiceProvider.php
 - └─ IntegrationServiceProvider.php
- └─ Services/
 - └─ Billing/
 - └─ StripeService.php
 - └─ InvoiceService.php
 - └─ Registrar/
 - └─ RegistrarInterface.php
 - └─ NameSiloRegistrar.php

```

├── NamecheapRegistrar.php
├── RegistrarResponseParser.php
├── Hosting/
│   ├── WhmService.php
│   └── CpanelPackageMapper.php
├── DNS/
│   ├── CloudflareService.php
│   └── DefaultDnsRecordBuilder.php
├── Provisioning/
│   ├── ProvisioningOrchestrator.php
│   ├── ProvisioningLogger.php
│   └── ProvisioningRetryService.php
├── Renewals/
│   ├── RenewalService.php
│   └── SuspensionService.php
├── bootstrap/
├── config/
│   ├── app.php
│   ├── auth.php
│   ├── billing.php
│   ├── cloudflare.php
│   ├── domain.php
│   ├── hosting.php
│   ├── mail.php
│   ├── queue.php
│   ├── services.php
│   ├── stripe.php
│   └── whm.php
├── database/
│   ├── migrations/
│   ├── seeders/
│   │   ├── DatabaseSeeder.php
│   │   ├── RoleSeeder.php
│   │   ├── ProductSeeder.php
│   │   └── HostingPackageSeeder.php
│   └── factories/
├── public/
│   ├── index.php
│   ├── assets/
│   └── images/
├── resources/

```

```
|
|
|└─ css/
|   └─ app.css
|
|└─ js/
|   └─ app.js
|
|└─ views/
|   └─ layouts/
|      └─ app.blade.php
|      └─ public.blade.php
|      └─ customer.blade.php
|      └─ admin.blade.php
|
|   └─ public/
|      └─ home.blade.php
|      └─ pricing.blade.php
|      └─ website-package.blade.php
|      └─ hosting.blade.php
|      └─ domain-search.blade.php
|      └─ contact.blade.php
|      └─ privacy-policy.blade.php
|      └─ terms.blade.php
|
|   └─ auth/
|      └─ login.blade.php
|      └─ register.blade.php
|      └─ forgot-password.blade.php
|
|   └─ checkout/
|      └─ cart.blade.php
|      └─ checkout.blade.php
|      └─ success.blade.php
|      └─ failed.blade.php
|
|   └─ customer/
|      └─ dashboard.blade.php
|      └─ domains/
|      └─ hosting/
|      └─ billing/
|      └─ website-projects/
|      └─ support/
|
|   └─ emails/
|      └─ order-confirmation.blade.php
|      └─ hosting-welcome.blade.php
|      └─ domain-registered.blade.php
|      └─ payment-failed.blade.php
|      └─ renewal-reminder.blade.php
|
|   └─ components/
```

```

├── pricing-card.blade.php
├── service-status-badge.blade.php
├── invoice-status-badge.blade.php
├── dns-record-table.blade.php
├── routes/
│   ├── web.php
│   ├── auth.php
│   ├── customer.php
│   ├── admin.php
│   ├── api.php
│   └── console.php
├── storage/
├── tests/
│   ├── Feature/
│   │   ├── CheckoutTest.php
│   │   ├── DomainSearchTest.php
│   │   ├── StripeWebhookTest.php
│   │   ├── ProvisioningTest.php
│   │   └── CustomerDashboardTest.php
│   └── Unit/
│       ├── NameSiloRegistrarTest.php
│       ├── WhmServiceTest.php
│       ├── CloudflareServiceTest.php
│       └── RenewalServiceTest.php
├── .env.example
├── artisan
├── composer.json
├── package.json
├── vite.config.js
└── README.md

```

5. Route Structure

5.1 Public Routes

GET	/	Home page
GET	/website-package	£200 bespoke website package
GET	/hosting	Hosting packages
GET	/domains	Domain search page
POST	/domains/search	Domain availability search
GET	/contact	Contact page

GET	/privacy-policy	Privacy policy
GET	/terms	Terms of use

5.2 Auth Routes

```
GET /login
POST /login
GET /register
POST /register
POST /logout
GET /forgot-password
POST /forgot-password
```

5.3 Checkout Routes

```
GET /cart
POST /cart/add
POST /cart/remove
GET /checkout
POST /checkout/start
GET /checkout/success
GET /checkout/cancel
```

5.4 Customer Dashboard Routes

```
GET /dashboard
GET /dashboard/domains
GET /dashboard/domains/{domain}
GET /dashboard/domains/{domain}/dns
POST /dashboard/domains/{domain}/dns
GET /dashboard/hosting
GET /dashboard/hosting/{hostingAccount}
GET /dashboard/billing
GET /dashboard/invoices/{invoice}
GET /dashboard/website-projects
GET /dashboard/website-projects/{project}
POST /dashboard/website-projects/{project}/intake
GET /dashboard/support
POST /dashboard/support
GET /dashboard/support/{ticket}
POST /dashboard/support/{ticket}/reply
```

5.5 Webhook Routes

```
POST /webhooks/stripe
```

5.6 Admin Routes

Admin routes should be handled mainly through Filament.

```
GET /admin
```

6. Database Schema

6.1 users

Purpose

Stores all customer and admin user accounts.

Fields

id	bigint primary key
name	varchar
email	varchar unique
email_verified_at	timestamp nullable
password	varchar
phone	varchar nullable
company_name	varchar nullable
billing_address_line_1	varchar nullable
billing_address_line_2	varchar nullable
billing_city	varchar nullable
billing_state	varchar nullable
billing_postcode	varchar nullable
billing_country	varchar nullable
stripe_customer_id	varchar nullable
is_admin	boolean default false
status	varchar default active
remember_token	varchar nullable
created_at	timestamp
updated_at	timestamp

Plain English

Each customer or admin is a user. A user can have many orders, domains, hosting accounts, subscriptions, invoices, website projects, and support tickets.

6.2 roles

Purpose

Stores user roles if you want more flexible permissions later.

Fields

id	bigint primary key
name	varchar unique
label	varchar
created_at	timestamp
updated_at	timestamp

Example Roles

```
admin
support
developer
customer
```

6.3 role_user

Purpose

Pivot table connecting users to roles.

Fields

id	bigint primary key
user_id	foreign key users.id
role_id	foreign key roles.id
created_at	timestamp
updated_at	timestamp

Relationship

One user can have multiple roles. One role can belong to multiple users.

6.4 products

Purpose

Stores sellable products such as website package, hosting plans, domain registration, and maintenance plans.

Fields

id	bigint primary key
name	varchar
slug	varchar unique
type	varchar
description	text nullable
is_active	boolean default true
sort_order	integer default 0
created_at	timestamp
updated_at	timestamp

Product Types

```
website_package
hosting
domain
maintenance
addon
```

Plain English

A product is something the customer can buy.

Examples:

- Complete Bespoke Website
- Starter Hosting
- Business Hosting
- Pro Hosting
- Domain Registration
- Website Maintenance

6.5 product_prices

Purpose

Stores prices for products.

Fields

id	bigint primary key
product_id	foreign key products.id
billing_cycle	varchar
currency	varchar default GBP
amount	decimal(10,2)
setup_fee	decimal(10,2) default 0
stripe_price_id	varchar nullable
is_active	boolean default true
created_at	timestamp
updated_at	timestamp

Billing Cycles

one_time
monthly
yearly

Plain English

A product can have multiple prices. For example, hosting can have monthly and yearly pricing.

6.6 hosting_packages

Purpose

Maps your website hosting packages to WHM package names.

Fields

id	bigint primary key
product_id	foreign key products.id
name	varchar
whm_package_name	varchar
disk_limit_mb	integer nullable
bandwidth_limit_mb	integer nullable
email_accounts_limit	integer nullable
database_limit	integer nullable
domain_limit	integer nullable
is_active	boolean default true
created_at	timestamp
updated_at	timestamp

Plain English

This table connects your public hosting package to the real WHM package.

Example:

Starter Hosting on your website maps to `planetic_starter` in WHM.

6.7 carts

Purpose

Stores customer carts before checkout.

Fields

<code>id</code>	<code>bigint primary key</code>
<code>user_id</code>	<code>foreign key users.id nullable</code>
<code>session_id</code>	<code>varchar nullable</code>
<code>currency</code>	<code>varchar default GBP</code>
<code>subtotal</code>	<code>decimal(10,2) default 0</code>
<code>discount_total</code>	<code>decimal(10,2) default 0</code>
<code>tax_total</code>	<code>decimal(10,2) default 0</code>
<code>total</code>	<code>decimal(10,2) default 0</code>
<code>status</code>	<code>varchar default active</code>
<code>created_at</code>	<code>timestamp</code>
<code>updated_at</code>	<code>timestamp</code>

Plain English

A cart holds selected products before payment.

6.8 cart_items

Purpose

Stores individual items inside the cart.

Fields

<code>id</code>	<code>bigint primary key</code>
<code>cart_id</code>	<code>foreign key carts.id</code>
<code>product_id</code>	<code>foreign key products.id</code>
<code>product_price_id</code>	<code>foreign key product_prices.id nullable</code>
<code>item_type</code>	<code>varchar</code>
<code>name</code>	<code>varchar</code>

domain_name	varchar nullable
quantity	integer default 1
unit_price	decimal(10,2)
total	decimal(10,2)
metadata	json nullable
created_at	timestamp
updated_at	timestamp

Item Types

```
website_package
hosting
domain_registration
domain_renewal
addon
maintenance
```

Plain English

If a customer buys a website package plus a domain, each item is stored separately in the cart.

6.9 orders

Purpose

Stores confirmed customer orders.

Fields

id	bigint primary key
user_id	foreign key users.id
order_number	varchar unique
status	varchar
payment_status	varchar
currency	varchar default GBP
subtotal	decimal(10,2)
discount_total	decimal(10,2) default 0
tax_total	decimal(10,2) default 0
total	decimal(10,2)
stripe_checkout_session_id	varchar nullable
stripe_payment_intent_id	varchar nullable
stripe_subscription_id	varchar nullable
paid_at	timestamp nullable
cancelled_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Order Statuses

```
pending
paid
provisioning
completed
failed
cancelled
refunded
manual_review
```

Plain English

An order is created when the customer checks out. The order tracks whether the customer paid and whether services were provisioned.

6.10 order_items

Purpose

Stores products included in an order.

Fields

id	bigint primary key
order_id	foreign key orders.id
product_id	foreign key products.id nullable
product_price_id	foreign key product_prices.id nullable
item_type	varchar
name	varchar
domain_name	varchar nullable
quantity	integer default 1
unit_price	decimal(10,2)
total	decimal(10,2)
metadata	json nullable
created_at	timestamp
updated_at	timestamp

Plain English

Each order can contain multiple items, such as:

- Website package
 - Domain registration
 - Hosting package
 - Maintenance plan
-

6.11 payments

Purpose

Stores payment attempts and successful payments.

Fields

id	bigint primary key
user_id	foreign key users.id
order_id	foreign key orders.id nullable
invoice_id	foreign key invoices.id nullable
provider	varchar default stripe
provider_payment_id	varchar nullable
provider_customer_id	varchar nullable
amount	decimal(10,2)
currency	varchar default GBP
status	varchar
failure_reason	text nullable
paid_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Payment Statuses

pending
succeeded
failed
refunded
partially_refunded
cancelled

Plain English

This records every payment event. If a payment fails, the reason is stored for support/admin review.

6.12 invoices

Purpose

Stores invoices for orders, subscriptions, and renewals.

Fields

id	bigint primary key
user_id	foreign key users.id

order_id	foreign key orders.id nullable
subscription_id	foreign key subscriptions.id nullable
invoice_number	varchar unique
stripe_invoice_id	varchar nullable
currency	varchar default GBP
subtotal	decimal(10,2)
tax_total	decimal(10,2) default 0
total	decimal(10,2)
amount_paid	decimal(10,2) default 0
amount_due	decimal(10,2) default 0
status	varchar
due_date	date nullable
paid_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Invoice Statuses

```

draft
open
paid
void
uncollectible
failed
refunded

```

Plain English

Invoices are shown to customers in the billing dashboard.

6.13 subscriptions

Purpose

Stores recurring services such as hosting, domain renewals, and maintenance plans.

Fields

id	bigint primary key
user_id	foreign key users.id
product_id	foreign key products.id
domain_id	foreign key domains.id nullable
hosting_account_id	foreign key hosting_accounts.id nullable
stripe_subscription_id	varchar nullable
status	varchar
billing_cycle	varchar
currency	varchar default GBP

amount	decimal(10,2)
current_period_start	date nullable
current_period_end	date nullable
next_renewal_date	date nullable
cancel_at_period_end	boolean default false
cancelled_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Subscription Statuses

```

active
past_due
cancelled
unpaid
paused
trialing
expired

```

Plain English

A subscription represents any recurring service. Hosting usually has a subscription. Maintenance has a subscription. Domain renewal may also be represented as a subscription if you want automatic yearly billing.

6.14 domains

Purpose

Stores all customer domains managed by Planetic Solution.

Fields

id	bigint primary key
user_id	foreign key users.id
order_id	foreign key orders.id nullable
domain_name	varchar unique
sld	varchar
tld	varchar
registrar	varchar
registrar_domain_id	varchar nullable
registrar_order_id	varchar nullable
status	varchar
registration_date	date nullable
expiry_date	date nullable
auto_renew	boolean default true
whois_privacy	boolean default true

registrar_lock	boolean default true
cloudflare_zone_id	foreign key cloudflare_zones.id nullable
nameservers	json nullable
last_synced_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Domain Statuses

pending
 available_checked
 registration_pending
 active
 expired
 renewal_due
 renewal_failed
 suspended
 cancelled
 failed
 manual_review

Plain English

Each domain belongs to a customer. It may be connected to a Cloudflare zone and one hosting account.

6.15 domain_contacts

Purpose

Stores domain registrant/admin/technical contact details used for domain registration.

Fields

id	bigint primary key
domain_id	foreign key domains.id
contact_type	varchar
first_name	varchar
last_name	varchar
company_name	varchar nullable
email	varchar
phone	varchar
address_line_1	varchar
address_line_2	varchar nullable
city	varchar
state	varchar nullable
postcode	varchar
country	varchar

created_at	timestamp
updated_at	timestamp

Contact Types

```
registrant
admin
technical
billing
```

Plain English

Registrars require contact details when registering domains. This table stores those details.

6.16 hosting_accounts

Purpose

Stores cPanel hosting accounts created through WHM.

Fields

id	bigint primary key
user_id	foreign key users.id
order_id	foreign key orders.id nullable
domain_id	foreign key domains.id nullable
hosting_package_id	foreign key hosting_packages.id
domain_name	varchar
whm_username	varchar unique
whm_account_id	varchar nullable
server_hostname	varchar nullable
server_ip	varchar
cpanel_url	varchar nullable
status	varchar
disk_limit_mb	integer nullable
bandwidth_limit_mb	integer nullable
created_on_whm_at	timestamp nullable
suspended_at	timestamp nullable
suspension_reason	text nullable
renewal_date	date nullable
last_synced_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Hosting Statuses

```
pending
creating
active
suspended
terminated
failed
manual_review
```

Plain English

This table stores the hosting account created inside Namecheap WHM.

One user can have many hosting accounts.

A hosting account usually belongs to one domain.

6.17 cloudflare_zones

Purpose

Stores Cloudflare zone information for each domain.

Fields

id	bigint primary key
user_id	foreign key users.id
domain_id	foreign key domains.id
zone_id	varchar unique
zone_name	varchar
status	varchar
name_servers	json nullable
ssl_mode	varchar nullable
always_use_https	boolean default true
created_on_cloudflare_at	timestamp nullable
last_synced_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Cloudflare Zone Statuses

```
pending
active
initializing
moved
```

```
deleted
failed
manual_review
```

Plain English

A Cloudflare zone represents the domain inside Cloudflare. It stores the Cloudflare zone ID and assigned nameservers.

6.18 dns_records

Purpose

Stores DNS records created or managed in Cloudflare.

Fields

id	bigint primary key
user_id	foreign key users.id
domain_id	foreign key domains.id
cloudflare_zone_id	foreign key cloudflare_zones.id
cloudflare_record_id	varchar nullable
type	varchar
name	varchar
content	text
ttl	integer nullable
proxied	boolean default false
priority	integer nullable
status	varchar default active
metadata	json nullable
created_at	timestamp
updated_at	timestamp

DNS Record Types

```
A
AAAA
CNAME
MX
TXT
NS
SRV
CAA
```

Plain English

This table mirrors DNS records created in Cloudflare. It allows your dashboard to display DNS records without calling Cloudflare on every page load.

6.19 provisioning_jobs

Purpose

Tracks every automation job required to complete an order.

Fields

id	bigint primary key
user_id	foreign key users.id
order_id	foreign key orders.id
job_type	varchar
status	varchar
attempts	integer default 0
max_attempts	integer default 3
started_at	timestamp nullable
completed_at	timestamp nullable
failed_at	timestamp nullable
error_message	text nullable
request_payload	json nullable
response_payload	json nullable
created_at	timestamp
updated_at	timestamp

Job Types

```
register_domain
create_cloudflare_zone
update_nameservers
create_hosting_account
create_dns_records
send_welcome_email
```

Job Statuses

```
pending
running
completed
failed
retrying
```

```
cancelled
manual_review
```

Plain English

Every automation step should be logged. If something fails, admin can see exactly which step failed and retry it.

6.20 website_projects

Purpose

Stores details for the £200 bespoke website development project.

Fields

id	bigint primary key
user_id	foreign key users.id
order_id	foreign key orders.id
domain_id	foreign key domains.id nullable
hosting_account_id	foreign key hosting_accounts.id nullable
project_number	varchar unique
status	varchar
business_name	varchar nullable
business_description	text nullable
industry	varchar nullable
pages_required	json nullable
brand_colours	varchar nullable
reference_websites	json nullable
special_requirements	text nullable
content_received	boolean default false
logo_received	boolean default false
target_launch_date	date nullable
launched_at	timestamp nullable
created_at	timestamp
updated_at	timestamp

Website Project Statuses

```
order_received
information_required
content_received
design_in_progress
review_required
revisions_in_progress
approved
launched
```

```
completed
on_hold
cancelled
```

Plain English

This table tracks the manual website development work after the customer pays for the package.

6.21 website_project_assets

Purpose

Stores uploaded files for website projects.

Fields

id	bigint primary key
website_project_id	foreign key website_projects.id
user_id	foreign key users.id
file_type	varchar
original_filename	varchar
stored_path	varchar
mime_type	varchar nullable
file_size	integer nullable
created_at	timestamp
updated_at	timestamp

File Types

```
logo
image
content_document
brand_guideline
other
```

Plain English

Customers can upload logos, images, content documents, or brand files.

6.22 support_tickets

Purpose

Stores customer support tickets.

Fields

id	bigint primary key
user_id	foreign key users.id
assigned_admin_id	foreign key users.id nullable
ticket_number	varchar unique
subject	varchar
category	varchar nullable
priority	varchar default normal
status	varchar default open
created_at	timestamp
updated_at	timestamp
closed_at	timestamp nullable

Ticket Statuses

open
pending
waiting_customer
resolved
closed

Priority

low
normal
high
urgent

Plain English

Customers can submit support requests. Admin can reply and manage ticket status.

6.23 support_ticket_messages

Purpose

Stores messages inside support tickets.

Fields

id	bigint primary key
support_ticket_id	foreign key support_tickets.id
user_id	foreign key users.id
message	text

is_internal_note	boolean default false
created_at	timestamp
updated_at	timestamp

Plain English

Each ticket can have multiple messages. Admins can also add internal notes that customers cannot see.

6.24 notification_logs

Purpose

Tracks emails and notifications sent by the system.

Fields

id	bigint primary key
user_id	foreign key users.id nullable
type	varchar
channel	varchar
recipient	varchar
subject	varchar nullable
status	varchar
sent_at	timestamp nullable
failed_at	timestamp nullable
error_message	text nullable
metadata	json nullable
created_at	timestamp
updated_at	timestamp

Plain English

This helps admin confirm whether important emails were sent.

6.25 audit_logs

Purpose

Stores important admin and system actions.

Fields

id	bigint primary key
user_id	foreign key users.id nullable
action	varchar
entity_type	varchar nullable

entity_id	bigint nullable
description	text nullable
old_values	json nullable
new_values	json nullable
ip_address	varchar nullable
user_agent	text nullable
created_at	timestamp

Plain English

This records who changed what. Useful for security, debugging, and accountability.

6.26 coupons

Purpose

Stores discount codes.

Fields

id	bigint primary key
code	varchar unique
description	text nullable
discount_type	varchar
discount_value	decimal(10,2)
max_uses	integer nullable
used_count	integer default 0
starts_at	timestamp nullable
expires_at	timestamp nullable
is_active	boolean default true
created_at	timestamp
updated_at	timestamp

Discount Types

fixed
percentage

6.27 coupon_redemptions

Purpose

Tracks coupon usage.

Fields

id	bigint primary key
coupon_id	foreign key coupons.id
user_id	foreign key users.id
order_id	foreign key orders.id
discount_amount	decimal(10,2)
created_at	timestamp
updated_at	timestamp

7. Entity Relationships Explained

User Relationships

A user can have:

- Many orders
- Many payments
- Many invoices
- Many subscriptions
- Many domains
- Many hosting accounts
- Many Cloudflare zones
- Many website projects
- Many support tickets

Order Relationships

An order belongs to one user.

An order can have:

- Many order items
- One or more payments
- One invoice
- One or more provisioning jobs
- One domain
- One hosting account
- One website project

Domain Relationships

A domain belongs to one user.

A domain can have:

- One Cloudflare zone
- Many DNS records
- One hosting account
- Many domain contacts
- One subscription for renewal

Hosting Account Relationships

A hosting account belongs to:

- One user
- One hosting package
- Usually one domain
- One subscription

Cloudflare Relationships

A Cloudflare zone belongs to:

- One user
- One domain

A Cloudflare zone has many DNS records.

Website Project Relationships

A website project belongs to:

- One user
- One order
- Optionally one domain
- Optionally one hosting account

A website project can have many uploaded assets.

Subscription Relationships

A subscription belongs to one user and one product.

It may be linked to:

- A hosting account
 - A domain
 - A maintenance plan
-

8. Core Workflows

8.1 Website Package Purchase Workflow

1. Customer visits website package page.
2. Customer searches domain.
3. Customer selects available domain.
4. Customer creates account.
5. Customer completes checkout.
6. Stripe confirms payment.
7. System creates order.
8. System creates website project.
9. System registers domain.
10. System creates Cloudflare zone.
11. System updates registrar nameservers.
12. System creates WHM hosting account.
13. System creates DNS records.
14. System sends order confirmation and welcome emails.
15. Customer fills website intake form.
16. Admin starts website development work.

8.2 Hosting-Only Purchase Workflow

1. Customer selects hosting package.
2. Customer enters existing domain or searches new domain.
3. Customer pays.
4. System creates WHM hosting account.
5. System creates Cloudflare zone if DNS management is selected.
6. System creates DNS records.
7. Customer receives hosting welcome email.
8. Customer manages hosting from dashboard.

8.3 Domain Registration Workflow

1. Customer searches domain.
2. System checks registrar API.
3. Customer selects domain.
4. Customer pays.
5. System registers domain.
6. System creates Cloudflare zone.
7. System updates nameservers.
8. System stores expiry date.
9. Customer sees domain in dashboard.

8.4 Renewal Workflow

1. Daily scheduler checks domains and hosting accounts due for renewal.
2. System sends reminder emails before renewal.
3. Stripe attempts renewal payment.

4. If payment succeeds:
 5. Subscription remains active
 6. Renewal date is extended
 7. Customer receives confirmation
 8. If payment fails:
 9. Customer receives failed payment email
 10. Admin is notified
 11. Grace period starts
 12. If unpaid after grace period:
 13. Hosting is suspended
 14. Domain renewal is flagged for manual review
 15. If customer pays later:
 16. Hosting is unsuspended
 17. Service continues
-

9. Provisioning Engine Design

9.1 Why Provisioning Must Use Queues

Provisioning involves multiple external APIs:

- Stripe
- Registrar
- Cloudflare
- WHM
- Email provider

External APIs can fail or be slow. Therefore, these steps should not run inside a normal web request.

Use queued jobs.

9.2 Provisioning Steps

For a complete website package:

```
ProvisionOrderJob
  → RegisterDomainJob
```

- CreateCloudflareZoneJob
- UpdateRegistrarNameserversJob
- CreateWhmHostingAccountJob
- CreateCloudflareDnsRecordsJob
- SendProvisioningCompletedEmailJob

9.3 Failure Handling

Every step must:

- Log request
- Log response
- Mark job status
- Retry if safe
- Stop if duplicate risk exists
- Notify admin if manual review is required

9.4 Idempotency

Every provisioning job must be idempotent.

This means if the job runs twice, it should not create duplicate domains, duplicate hosting accounts, or duplicate DNS records.

Before creating anything, check if it already exists in the database and/or external provider.

10. API Integration Design

10.1 Stripe Integration

Required Events

Handle these Stripe webhook events:

```
checkout.session.completed
payment_intent.succeeded
payment_intent.payment_failed
invoice.paid
invoice.payment_failed
customer.subscription.created
customer.subscription.updated
customer.subscription.deleted
```

Rules

- Always verify Stripe webhook signatures.

- Never trust frontend payment success alone.
- Only provision services after webhook confirmation.
- Store all Stripe IDs in database.
- Webhook endpoint must be publicly accessible over HTTPS.

10.2 Registrar Integration

Use `RegistrarInterface` so you can switch between NameSilo and Namecheap.

Example interface methods:

```
interface RegistrarInterface
{
    public function checkAvailability(string $domain): array;

    public function registerDomain(array $data): array;

    public function renewDomain(string $domain, int $years = 1): array;

    public function getDomainInfo(string $domain): array;

    public function updateNameservers(string $domain, array $nameservers):
array;
}
```

This design means your application is not locked into one registrar.

10.3 WHM Integration

WHM service should support:

```
createAccount
suspendAccount
unsuspendAccount
terminateAccount
listAccounts
getAccountInfo
changePackage
```

Important:

- WHM API calls must run from backend only.
- WHM token must never be exposed to frontend.
- Account usernames must be generated safely.
- Domain duplication must be checked before account creation.

10.4 Cloudflare Integration

Cloudflare service should support:

```
createZone
getZone
listZones
createDnsRecord
updateDnsRecord
deleteDnsRecord
setSslMode
enableAlwaysUseHttps
purgeCache
```

Default DNS records:

A	@	server_ip	proxied
CNAME	www	domain.com	proxied
A	mail	server_ip	DNS only
MX	@	mail.domain.com	DNS only
TXT	@	SPF	DNS only
TXT	_dmarc	DMARC	DNS only

Do not proxy:

```
mail
webmail
cpanel
whm
ftp
MX records
SPF
DKIM
DMARC
```

11. Environment Variables

Create a strong `.env.example` file.

```
APP_NAME="Planetic Solution"
APP_ENV=local
APP_KEY=
APP_DEBUG=true
APP_URL=http://localhost:8000
```

```
LOG_CHANNEL=stack
LOG_LEVEL=debug

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=planetic_platform
DB_USERNAME=root
DB_PASSWORD=

CACHE_STORE=database
SESSION_DRIVER=database
QUEUE_CONNECTION=database

MAIL_MAILER=smtp
MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=
MAIL_PASSWORD=
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=no-reply@planeticsolution.com
MAIL_FROM_NAME="Planetic Solution"

STRIPE_PUBLIC_KEY=
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
STRIPE_CURRENCY=gbp

BILLING_GRACE_PERIOD_DAYS=7
RENEWAL_REMINDER_DAYS_BEFORE=30,14,7,3,1

DOMAIN_REGISTRAR=namesilo

NAMESILO_API_KEY=
NAMESILO_API_ENDPOINT=https://www.namesilo.com/api
NAMESILO_SANDBOX=false

NAMECHEAP_API_USER=
NAMECHEAP_API_KEY=
NAMECHEAP_USERNAME=
NAMECHEAP_CLIENT_IP=
NAMECHEAP_API_ENDPOINT=https://api.namecheap.com/xml.response
NAMECHEAP_SANDBOX=false

WHM_HOST=
WHM_PORT=2087
WHM_USERNAME=
WHM_API_TOKEN=
WHM_VERIFY_SSL=true
WHM_DEFAULT_PACKAGE=planetic_starter
```

```
WHM_SERVER_IP=
WHM_CPANEL_LOGIN_URL=

CLOUDFLARE_API_TOKEN=
CLOUDFLARE_ACCOUNT_ID=
CLOUDFLARE_DEFAULT_SSL_MODE=full
CLOUDFLARE_ALWAYS_USE_HTTPS=true
CLOUDFLARE_PROXIED_WEBSITE_RECORDS=true

DEFAULT_DNS_TTL=1
DEFAULT_MAIL_SERVER_HOSTNAME=mail
DEFAULT_DMARC_POLICY=none

ADMIN_EMAIL=admin@planeticsolution.com
SUPPORT_EMAIL=support@planeticsolution.com

FILESYSTEM_DISK=local

APP_TIMEZONE=Europe/London
```

Notes About Environment Variables

APP_KEY

Must be generated using:

```
php artisan key:generate
```

APP_DEBUG

Must be false in production.

```
APP_DEBUG=false
```

Stripe

Use test keys in development and live keys in production.

Never mix test and live webhooks.

WHM

Use WHM API token, not root password.

Restrict the token to required permissions where possible.

Cloudflare

Use a scoped API token with access only to required zone/DNS permissions.

Do not use global API key if avoidable.

Registrar

Use sandbox/test mode during development if provider supports it.

Queue

For MVP:

```
QUEUE_CONNECTION=database
```

For scale:

```
QUEUE_CONNECTION=redis
```

12. Configuration Files

12.1 config/whm.php

```
return [  
    'host' => env('WHM_HOST'),  
    'port' => env('WHM_PORT', 2087),  
    'username' => env('WHM_USERNAME'),  
    'token' => env('WHM_API_TOKEN'),  
    'verify_ssl' => env('WHM_VERIFY_SSL', true),  
    'default_package' => env('WHM_DEFAULT_PACKAGE', 'planetic_starter'),  
    'server_ip' => env('WHM_SERVER_IP'),  
];
```

12.2 config/cloudflare.php

```
return [  
    'api_token' => env('CLOUDFLARE_API_TOKEN'),  
    'account_id' => env('CLOUDFLARE_ACCOUNT_ID'),  
    'default_ssl_mode' => env('CLOUDFLARE_DEFAULT_SSL_MODE', 'full'),  
    'always_use_https' => env('CLOUDFLARE_ALWAYS_USE_HTTPS', true),  
    'proxied_website_records' => env('CLOUDFLARE_PROXIED_WEBSITE_RECORDS',
```

```
true),  
];
```

12.3 config/domain.php

```
return [  
    'default_registrar' => env('DOMAIN_REGISTRAR', 'namesilo'),  
  
    'namesilo' => [  
        'api_key' => env('NAMESILO_API_KEY'),  
        'endpoint' => env('NAMESILO_API_ENDPOINT', 'https://www.namesilo.com/  
api'),  
        'sandbox' => env('NAMESILO_SANDBOX', false),  
    ],  
  
    'namecheap' => [  
        'api_user' => env('NAMECHEAP_API_USER'),  
        'api_key' => env('NAMECHEAP_API_KEY'),  
        'username' => env('NAMECHEAP_USERNAME'),  
        'client_ip' => env('NAMECHEAP_CLIENT_IP'),  
        'endpoint' => env('NAMECHEAP_API_ENDPOINT'),  
        'sandbox' => env('NAMECHEAP_SANDBOX', false),  
    ],  
];
```

12.4 config/billing.php

```
return [  
    'currency' => env('STRIPE_CURRENCY', 'gbp'),  
    'grace_period_days' => env('BILLING_GRACE_PERIOD_DAYS', 7),  
    'renewal_reminder_days_before' => explode(',',  
env('RENEWAL_REMINDER_DAYS_BEFORE', '30,14,7,3,1')),  
];
```

13. Security Architecture

13.1 Authentication

Use Laravel authentication with:

- Email/password login
- Email verification
- Password reset
- Secure password hashing
- Session-based authentication

13.2 Authorization

Use Laravel policies for customer-owned records.

Customers must only access their own:

- Domains
- Hosting accounts
- Invoices
- Subscriptions
- Website projects
- Support tickets

Admins can access all records.

13.3 API Secrets

Never expose these to frontend:

- Stripe secret key
- Stripe webhook secret
- WHM token
- Cloudflare token
- Registrar API key
- SMTP password

13.4 Webhook Security

Stripe webhook endpoint must:

- Verify signature
- Reject invalid signatures
- Log event ID
- Avoid duplicate processing
- Return 200 only after safe handling

13.5 Rate Limiting

Apply rate limits to:

- Login
- Register
- Domain search
- Checkout start
- Support ticket creation
- Stripe webhook endpoint

13.6 Audit Logging

Log these actions:

- Admin login
 - Admin changes customer details
 - Admin suspends hosting
 - Admin retries provisioning
 - Admin cancels service
 - Admin changes renewal date
 - Admin changes invoice/payment status
-

14. Scheduler and Cron Jobs

14.1 Required Cron Entry

Production server should run Laravel scheduler every minute:

```
* * * * * cd /path/to/project && php artisan schedule:run >> /dev/null 2>&1
```

14.2 Scheduled Tasks

In `routes/console.php`:

```
Schedule::command('renewals:check')->dailyAt('09:00');  
Schedule::command('domains:sync')->dailyAt('02:00');  
Schedule::command('hosting:sync')->dailyAt('03:00');  
Schedule::command('provisioning:retry-failed')->hourly();
```

14.3 Renewal Reminder Schedule

Send reminders before renewal:

```
30 days before  
14 days before  
7 days before  
3 days before  
1 day before  
On renewal day
```

15. Queue Worker Configuration

15.1 MVP Database Queue

Run worker:

```
php artisan queue:work --tries=3 --timeout=120
```

15.2 Production with Supervisor

Example Supervisor config:

```
[program:planetic-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /path/to/project/artisan queue:work --sleep=3 --tries=3 --
timeout=120
autostart=true
autorestart=true
numprocs=2
redirect_stderr=true
stdout_logfile=/path/to/project/storage/logs/worker.log
```

15.3 Queue Separation Later

Later, use separate queues:

```
default
payments
provisioning
emails
renewals
```

16. MVP Build Sequence

Phase 1: Foundation

1. Install Laravel.
2. Configure database.
3. Set up authentication.
4. Create public pages.
5. Create products and prices.
6. Create admin panel.

7. Create customer dashboard shell.

Phase 2: Checkout

1. Create cart.
2. Add product selection.
3. Add domain search.
4. Add Stripe Checkout.
5. Add Stripe webhook.
6. Store orders and payments.

Phase 3: Provisioning

1. Build registrar service.
2. Build WHM service.
3. Build Cloudflare service.
4. Build provisioning jobs.
5. Add provisioning logs.
6. Add admin retry function.

Phase 4: Customer Dashboard

1. Show domains.
2. Show hosting accounts.
3. Show invoices.
4. Show subscriptions.
5. Show renewal dates.
6. Add website intake form.
7. Add support tickets.

Phase 5: Renewals

1. Add renewal reminder scheduler.
2. Add failed payment handling.
3. Add hosting suspension rules.
4. Add reactivation after payment.
5. Add admin renewal alerts.

17. Testing Strategy

17.1 Unit Tests

Test:

- Domain availability parsing
- Registrar API response parsing
- WHM username generation
- Cloudflare DNS record builder

- Renewal date calculation
- Price calculation

17.2 Feature Tests

Test:

- Customer registration
- Domain search
- Cart add/remove
- Checkout creation
- Stripe webhook processing
- Order creation
- Provisioning job creation
- Dashboard access permissions

17.3 Integration Tests

Use sandbox/test modes where available.

Test:

- Stripe checkout
- Stripe webhooks
- Registrar domain availability
- Cloudflare zone creation
- WHM test account creation

Do not test live domain registration repeatedly because it may cost real money.

18. Deployment Notes

18.1 Production Checklist

Before launch:

- APP_DEBUG=false
- HTTPS enabled
- Correct APP_URL
- Live Stripe keys added
- Live Stripe webhook configured
- WHM API token added
- Cloudflare API token added
- Registrar API key added
- Queue worker running
- Scheduler cron running
- Database backups enabled
- Error logs monitored

- Admin email configured
- Test purchase completed
- Test provisioning completed
- Test failed payment flow completed

18.2 Backup Strategy

Back up:

- MySQL database daily
- Uploaded website project files daily
- `.env` securely outside public web root
- Storage folder
- Logs for debugging

Do not store plain API secrets in database backups if avoidable.

18.3 Logging

Use separate logs for:

- Laravel app errors
- Stripe webhooks
- Registrar API
- WHM API
- Cloudflare API
- Provisioning jobs
- Email delivery

19. Key Technical Rules for Developers

1. Do not call third-party APIs from controllers directly.
 2. All external integrations must use service classes.
 3. All provisioning must run through queues.
 4. Every provisioning step must be logged.
 5. Every provisioning job must be safe to retry.
 6. Never expose secret API keys to frontend.
 7. Never provision services based only on frontend payment success.
 8. Provision only after verified Stripe webhook.
 9. Keep registrar integration replaceable.
 10. Keep customer data isolated by user ID.
 11. Use clear status fields for every service.
 12. Admin must be able to manually retry failed automation.
-

20. Final Architecture Recommendation

The best architecture for Planetic Solution is:

```
Laravel monolithic application
+ MySQL database
+ Laravel Blade / Tailwind CSS frontend
+ Filament admin panel
+ Stripe Checkout and Billing
+ Stripe webhooks
+ NameSilo registrar API
+ Namecheap WHM API
+ Cloudflare DNS API
+ Laravel queues
+ Laravel scheduler
+ SMTP transactional email provider
```

This gives you a professional, scalable, and realistic platform without overengineering the MVP.

The most important architectural decision is to keep the project modular:

- Registrar can be changed later.
- Hosting provider can be changed later.
- DNS provider can be changed later.
- Payment provider can be expanded later.
- Frontend can be upgraded later.

Start with a strong Laravel monolith. Build the integrations cleanly. Use queues for automation. Use a proper admin panel. Keep all statuses visible. Make every failed automation step retryable.